PROG1384

Développement Web côté serveur



ccnb.ca

Introduction à Ajax

Appels asynchrones avec XMLHttpRequest

Objectif du cours : comprendre comment une page web peut demander une ressource sans recharger toute la page.

Le problème de départ

Sur une page web traditionnelle :

1. l'utilisateur clique ou soumet un formulaire;
2. le navigateur envoie une requête au serveur;
3. le serveur répond avec une nouvelle page complète;
4. toute la page est rechargée.

Ajax permet de mettre à jour seulement une partie de la page.

Ajax en une phrase

Ajax permet à JavaScript de faire une requête au serveur en arrière-plan.

La page reste ouverte pendant que la requête se fait.

Quand la réponse arrive, JavaScript peut modifier le contenu affiché dans la page avec le DOM.

Pourquoi dit-on asynchrone?

Un appel asynchrone ne bloque pas la page.

Pendant que le navigateur attend la réponse :

- l'utilisateur peut continuer à utiliser la page;
- JavaScript peut exécuter d'autres instructions;
- la réponse sera traitée lorsqu'elle sera prête.

Les morceaux importants

Une interaction Ajax de base contient trois parties :

1. un déclencheur;
2. une requête faite par JavaScript;
3. une mise à jour de la page.

Exemples de déclencheurs : clic, touche du clavier, minuterie, changement dans un champ.

Le rôle de chaque technologie

HTML structure la page.

CSS présente la page.

JavaScript déclenche l'appel et met la page à jour.

Le serveur retourne une réponse.

Le DOM permet de modifier seulement une section de la page.

Exemple 1 : lire un fichier texte

Pour commencer, le serveur n'a même pas besoin d'exécuter du PHP.

On peut demander un simple fichier texte.

Cet exemple montre qu'un appel Ajax peut récupérer une ressource ordinaire sur le serveur.

Fichier texte : message.txt

```
Bonjour!

Ce message vient d'un fichier texte sur le serveur.
La page n'a pas été rechargée.
```

Le fichier peut être placé dans le même dossier que la page HTML.

Page HTML de départ

```
<h1>Exemple Ajax</h1>

<button id="btnCharger">Charger le message</button>

<div id="resultat"></div>
```

Le bouton déclenchera l'appel Ajax.

Le résultat sera affiché dans la balise `div`.

JavaScript : créer l'objet XHR

```
const bouton = document.querySelector("#btnCharger");
const resultat = document.querySelector("#resultat");

bouton.addEventListener("click", function () {
  const xhr = new XMLHttpRequest();
});
```

`XMLHttpRequest` est l'objet JavaScript qui fait le lien entre la page et le serveur.

JavaScript : préparer la requête

```
bouton.addEventListener("click", function () {
  const xhr = new XMLHttpRequest();

  xhr.open("GET", "message.txt", true);
});
```

`GET` indique qu'on veut lire une ressource.

`message.txt` est la ressource demandée.

`true` indique que l'appel est asynchrone.

JavaScript : recevoir la réponse

```
bouton.addEventListener("click", function () {
  const xhr = new XMLHttpRequest();

  xhr.open("GET", "message.txt", true);

  xhr.onload = function () {
    resultat.textContent = xhr.responseText;
  };

  xhr.send();
});
```

`responseText` contient le texte reçu du serveur.

Vérifier si tout va bien

```
xhr.onload = function () {
  if (xhr.status === 200) {
    resultat.textContent = xhr.responseText;
  } else {
    resultat.textContent = "Erreur de chargement.";
  }
};
```

`status` contient le code HTTP retourné par le serveur.

Le code `200` signifie que la requête a réussi.

Version complète

```
const bouton = document.querySelector("#btnCharger");
const resultat = document.querySelector("#resultat");

bouton.addEventListener("click", function () {
  const xhr = new XMLHttpRequest();

  xhr.open("GET", "message.txt", true);

  xhr.onload = function () {
    if (xhr.status === 200) {
      resultat.textContent = xhr.responseText;
    } else {
      resultat.textContent = "Erreur de chargement.";
    }
  };

  xhr.send();
});
```

Exemple 2 : PHP retourne du texte

Une fois le principe compris, le serveur peut exécuter du PHP.

Fichier `message.php` :

```php
<?php

echo 'Bonjour! Ce texte vient maintenant de PHP.';
```

Dans JavaScript, on change seulement l'adresse demandée :

```
xhr.open("GET", "message.php", true);
```

Ce qu'il faut retenir

Ajax permet de demander une ressource sans recharger toute la page.

`XMLHttpRequest` prépare, envoie et reçoit la requête.

L'appel asynchrone permet à la page de rester utilisable.

La réponse peut ensuite être affichée dans une partie précise de la page.

La prochaine étape sera de faire communiquer JavaScript avec un vrai script PHP plus utile.